



Structures de Données Python

Maîtriser les fondamentaux pour la Data Science & le Machine Learning

🎯 OBJECTIF GÉNÉRAL & PARCOURS D'APPRENTISSAGE

Cette série a pour objectif de maîtriser les principales structures de données Python avant d'aborder **NumPy**, **Pandas**, **Data Science** et **Machine Learning**.

LISTES



DICTIONNAIRES



ENSEMBLES



DATA SCIENCE

Compétences ciblées : stocker accéder modifier parcourir rechercher filtrer compter
trier agréger analyser transformer

🟢 PARTIE 1 — LES LISTES

Challenge 1 — Manipulation fondamentale

NIVEAU 1 — FONDAMENTAL

🎯 **Objectif** : Maîtriser l'indexation, les indices négatifs, le slicing et la modification d'une liste.

DONNÉES

```
fruits = ["Pomme", "Banane", "Orange", "Fraise", "Mangue", "Kiwi"]
```

TRAVAIL À FAIRE

A. Accéder aux données

- Afficher la liste complète.
- Afficher le premier élément.
- Afficher le dernier élément.
- Afficher le troisième élément.

B. Utiliser le slicing

- Afficher les trois premiers éléments.
- Afficher les trois derniers éléments.
- Afficher un élément sur deux.

C. Modifier les données

- Remplacer "Orange" par "Ananas".
- Afficher la liste après modification.

⚠ **Contraintes** : Ne pas recréer une deuxième liste.
Utiliser obligatoirement les indices et le slicing.

🎯 **Notions** : `index`, `-1`, `slicing [:]`, `modification`.

💡 **Remarque** : En Python, le premier élément possède l'indice 0.

Challenge 2 — Ajouter et supprimer

NIVEAU 1 — DYNAMIQUE

 **Objectif** : Savoir modifier dynamiquement une liste.

DONNÉES

```
langages = ["Python", "Java", "JavaScript", "C++"]
```

TRAVAIL À FAIRE

A. Ajouter

- Ajouter `"PHP"` à la fin.
- Ajouter `"SQL"` à la fin.
- Insérer `"C"` en deuxième position.

B. Supprimer & Analyser

- Supprimer `"Java"`.
- Supprimer le dernier élément.
- Afficher la liste finale et le nombre d'éléments.

 **Notions** : `append()`, `insert()`, `remove()`, `pop()`, `len()`.

 **Remarque** : Bien comprendre la différence entre `remove("Java")` (supprime par valeur) et `pop()` (supprime par indice / dernier élément).

Challenge 3 — Analyse des notes

NIVEAU 2 — ANALYSE

 **Objectif** : Parcourir une liste et produire des statistiques simples.

DONNÉES

```
notes = [12, 4, 14, 11, 18, 13, 7, 10, 5, 9, 15, 8, 14, 16]
```

TRAVAIL À FAIRE

1. Afficher toutes les notes.
2. Calculer la moyenne.
3. Créer une liste contenant les notes supérieures à la moyenne.
4. Créer une liste contenant les notes inférieures à la moyenne.
5. Trouver la meilleure note et la plus mauvaise note.
6. Compter le nombre de notes supérieures ou égales à 10 et calculer le pourcentage de réussite.

 **Notions** : `for`, conditions, `sum()`, `len()`, `min()`, `max()`, filtrage, compteur.

 **Transition clé** : Liste → Parcours → Calcul → Filtrage → Analyse.

Challenge 4 — Filtrer des données

NIVEAU 2 — FILTRAGE

🎯 **Objectif** : Sélectionner des données selon des conditions précises.

DONNÉES

```
temperatures = [18, 25, 31, 14, 27, 35, 22, 19, 30, 12, 28]
```

TRAVAIL À FAIRE (UTILISANT `FOR`, `IF`, `APPEND()`)

- Créer une liste des températures > 25 .
- Créer une liste des températures ≤ 25 .
- Créer une liste des températures comprises entre 20 et 30 inclus (remarque : `20 <= temp <= 30`).
- Compter le nombre de températures supérieures à 30.

Challenge 5 — Trier et organiser les données

NIVEAU 2 — TRI

🎯 **Objectif** : Comprendre le tri et la différence entre modification sur place et copie.

DONNÉES

```
scores = [45, 12, 78, 34, 90, 23, 67, 56, 89, 10]
```

TRAVAIL À FAIRE

- Afficher la liste originale, puis créer une copie (`copy()`).
- Trier la première copie dans l'ordre croissant et une deuxième copie dans l'ordre décroissant.
- Afficher les deux résultats ainsi que les trois meilleurs scores.

⚠ **Attention** : `scores.copy() != scores` et `sort()` modifie la liste sur place !

Challenge 6 — Rechercher un élément

NIVEAU 3 — RECHERCHE ALGORITHMIQUE

🎯 **Objectif** : Implémenter soi-même une recherche dans une liste sans utiliser `list.index()`.

Créer la fonction `rechercheElement(element, liste)` qui retourne l'indice si trouvé, sinon `False`.

```
L = [10, 20, 30, 40, 50]
rechercheElement(30, L) # Retourne: 2
rechercheElement(100, L) # Retourne: False
```

Challenge 7 — Compter les occurrences

NIVEAU 3 — ALGORITHMIQUE

🎯 **Objectif** : Compter les occurrences sans utiliser `count()`.

Données : `L = [7, 23, 5, 23, 7, 19, 23, 12, 29]`. Créer `compterOccurrences(element, liste)`.

```
compterOccurrences(23, L) # 3
compterOccurrences(7, L) # 2
compterOccurrences(100, L) # 0
```

💡 **Pattern classique** : `compteur = 0` → `for element in liste:` → `if cond: compteur += 1`

Challenge 8 — Fréquence des éléments

NIVEAU 3 — FRÉQUENCES

🎯 **Objectif** : Déterminer la fréquence de chaque élément unique (sans utiliser `Counter` dans un premier temps).

Données : `L = [7, 23, 5, 23, 7, 19, 23, 12, 29, 7, 5]`

RÉSULTAT ATTENDU

```
7 → 3 | 23 → 3 | 5 → 2 | 19 → 1 | 12 → 1 | 29 → 1
```

★ **Remarque** : Prépare directement aux statistiques, NLP et au Machine Learning.

Challenge 9 — Texte → Listes → Comparaison

NIVEAU 4 — DONNÉES COMPLEXES

🎯 **Objectif** : Transformer des données textuelles et les analyser.

- A partir de 2 textes, transformer chaque texte en liste de mots (`split()`).
- Convertir en minuscules et éliminer les mots de 3 lettres ou moins.
- Comparer les listes et identifier les mots communs.

Challenge 10 — Données hétérogènes

NIVEAU 4 — STRUCTURE

🎯 **Objectif** : Comprendre les limites d'une liste multi-types.

```
donnees = ["Omar", 25, "Casablanca", 15.5, True]
```

- Afficher chaque élément et son type (`type()`).
- Compter le nombre de chaînes, nombres et booléens.
- Créer une nouvelle liste contenant uniquement les nombres.

⚠️ **Evolution** : Liste → Dictionnaire → Liste de dictionnaires → DataFrame (Pandas).

Challenge 11 — Compréhension de listes (List Comprehensions)

NIVEAU 5 — PYTHONIC CODE

🎯 **Objectif** : Réécrire des traitements de manière concise.

Données : `nombres = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`

Créer en une seule ligne : une liste des carrés, une liste des nombres pairs, et une liste des nombres > 5.

PARTIE 2 — LES DICTIONNAIRES

💡 CHANGEMENT DE LOGIQUE

Avec une liste `["Omar", 22, "Casablanca"]`, on manipule des **positions (indices)**.

Avec un dictionnaire `{"nom": "Omar", "age": 22, "ville": "Casablanca"}`, on manipule des **clés explicites**.

Challenge 1 & 2 — Base & Modification de Dictionnaires

NIVEAU 1 — FONDAMENTAL

Challenge 1 : Etudiant

```
etudiant = {
    "nom": "Omar", "age": 22,
    "ville": "Casablanca", "note": 15
}
```

- Afficher nom, âge et note.
- Modifier la note à 17.
- Ajouter `"formation": "IA"`.

Challenge 2 : Produit

```
produit = {
    "nom": "Ordinateur", "prix": 8500,
    "stock": 12, "categorie":
    "Informatique"
}
```

- Modifier le prix à 7900.
- Ajouter `"marque": "Lenovo"` et `"disponible": True`.
- Supprimer `"stock"` avec `del` et `"categorie"` avec `pop()`.

Challenge 3 & 4 — Parcourir, Analyser et Filtrer

NIVEAU 2 — PARCOURS & FILTRAGE

DONNÉES CHALLENGE 3

```
notes = {"Python": 15, "SQL": 13, "JavaScript": 17, "Git": 14, "Linux": 12}
```

- Afficher toutes les matières (`keys()`), toutes les notes (`values()`), et les paires (`items()`).
- Calculer la moyenne, la meilleure et la plus mauvaise note.

DONNÉES CHALLENGE 4

```
notes_etudiants = {"Omar": 15, "Sara": 8, "Yassine": 17, "Imane": 11, "Hamza": 6, "Nadia": 14}
```

- Séparer les étudiants ayant une note ≥ 10 et < 10 .
- Calculer le pourcentage de réussite et identifier le meilleur étudiant.

★ **Pattern fondamental** : `for cle, valeur in dictionnaire.items():`

Challenge 5 & 6 — Construction et Imbrication

NIVEAU 3 — STRUCTURATION

Challenge 5 : Associer dynamiquement 2 listes `noms = ["Python", "SQL", "Pandas", "NumPy"]` et `niveaux = [5, 4, 3, 4]` sans le faire manuellement.

Challenge 6 : Dictionnaire imbriqué

```
etudiant = {
    "nom": "Omar", "age": 22,
    "formation": {"nom": "Développement IA", "niveau": "Avancé", "duree": 12}
}
```

- Accéder au nom de la formation et modifier le niveau en `"Expert"`.
- Ajouter une clé `"technologies"` contenant une liste : `["Python", "SQL", "Pandas", "Machine Learning"]`.

Challenge 7 & 8 — Liste de Dictionnaires & Mini-Analyse

NIVEAU 4 — ANALYSE DE DONNÉES TABULAIRES

Challenge 7 : Liste d'étudiants

```
etudiants = [
    {"nom": "Omar", "age": 22, "note": 15},
    {"nom": "Sara", "age": 21, "note": 17},
    {"nom": "Yassine", "age": 23, "note": 9},
    {"nom": "Imane", "age": 20, "note": 13},
    {"nom": "Hamza", "age": 24, "note": 7}
]
```

Afficher les étudiants admis/échec, la moyenne de classe et l'étudiant ayant la meilleure note.

Challenge 8 : Mini-analyse de ventes

```
ventes = [
    {"produit": "PC", "categorie": "Informatique", "prix": 8000, "quantite": 2},
    {"produit": "Souris", "categorie": "Accessoire", "prix": 150, "quantite": 10},
    {"produit": "Clavier", "categorie": "Accessoire", "prix": 300, "quantite": 5},
    {"produit": "PC", "categorie": "Informatique", "prix": 8000, "quantite": 1},
    {"produit": "Écran", "categorie": "Informatique", "prix": 2500, "quantite": 3}
]
```

CALCULS DEMANDÉS

- Nombre total de ventes & Chiffre d'affaires (CA) total.
- Produit le plus cher & Quantité totale vendue.
- Chiffre d'affaires par produit & Nombre de produits par catégorie.

★ **Processus Analyste** : Données brutes → Parcours → Transformation → Agrégation → Résultats

PARTIE 3 — LES ENSEMBLES (SETS)

Challenge 1 — Mots communs entre deux textes

NIVEAU AVANCÉ — TRAITEMENT DE TEXTE

 **Objectif** : Découvrir la puissance des ensembles (sets) pour trouver l'intersection entre deux textes réels.

Règle : Extraire les mots en commun entre `text_1` et `text_2` en conservant uniquement les mots de **plus de 3 lettres**.

TEXTES D'ENTRÉE

Texte 1 — Régression Linéaire

Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable you want to predict is called the dependent variable. The variable you are using to predict the other variable's value is called the independent variable. This form of analysis estimates the

Texte 2 — Régression Logistique

Logistic regression is a supervised machine learning algorithm widely used for binary classification tasks, such as identifying whether an email is spam or not and diagnosing diseases by assessing the presence or absence of specific conditions based on patient test results. This approach utilizes the logistic

 **Notions clés** : `set()` , `intersection (&)` , nettoyage de chaînes, filtrage par longueur.